Prague University of Economics and Business
Faculty of Informatics and Statistics

A Data Integration Framework for Enterprise Application Security

MASTER'S THESIS

Study program: Applied Data Analytics and Artificial Intelligence
Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus
Supervisor: Ing. Aleš Kotuč

Prague, month YYYY

Acknowledgements

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

Abstract

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

Keywords

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

Contents

List of Figures

List of Tables

List of source codes

Introduction

Motivation

Objective

Methodology

Structure

1. Analysis and Requirements

This chapter surveys the literature and analyzes the functional domains relevant to building an application security data platform. Each section covers one domain, combining theoretical foundations with practical tool and Application Programming Interface (API) analysis. The focus is on reviewing existing knowledge, observing source system interfaces, data models, and integration patterns, not on prescribing a solution. Technology choices for the target platform are deferred to Chapter 2 and Chapter 3.

Section 1.1 establishes the business context underlying all security findings. Section 1.2 and Section 1.3 examine the software development and security tooling landscapes as data sources. Section 1.4 covers the data engineering patterns that underpin the framework's design. Finally, Section 1.5 surveys existing solutions and identifies the gap this thesis addresses.

1.1 Application Inventory

The application inventory is the foundational data source for any application security data platform. It establishes the business context for all security findings. Without a reliable catalog of business applications and their mapping to technical assets, a vulnerability discovered in a repository or a running service cannot be attributed to an owner, assessed for business impact, or prioritized against organizational risk criteria.

1.1.1 Application Portfolio Management

Enterprise organizations maintain catalogs of their business applications with metadata such as application name, description, ownership information, lifecycle status, and relationships to other assets (AXELOS, 2019).

These registries also capture security-relevant attributes. Information security is commonly assessed along three dimensions known as the CIA triad: confidentiality (preventing unauthorized disclosure), integrity (preventing unauthorized modification), and availability (ensuring authorized access) (Stallings & Brown, 2024). Business impact assessments evaluate all three CIA dimensions to produce criticality ratings, commonly structured as tiers: Tier 1 for mission-critical applications, Tier 2 for important supporting systems, and Tier 3 for non-critical internal tools. Separately, regulatory scope flags identify which compliance frameworks (e.g., PCI DSS, GDPR, SOX) apply to a given application.

The most important attribute from a data integration perspective is the mapping between business applications and their technical assets. A single business application may span

multiple source code repositories, microservices, infrastructure components, and deployment environments. This mapping serves as the glue connecting technical security findings to business context. When the framework ingests a vulnerability from a Static Application Security Testing tool associated with a specific repository, the application inventory mapping determines which business application is affected, who owns it, and how critical it is. This enables the risk-aware prioritization that stakeholders require.

1.1.2 Configuration Management Databases

Organizations typically maintain their application inventories in Configuration Management Databases (CMDBs) such as ServiceNow, in custom-built internal catalogs, or in a combination of both. A CMDB organizes all managed assets as Configuration Items (CIs), each with attributes describing its type, status, and ownership (AXELOS, 2019).

ServiceNow is the leading CMDB platform in enterprise environments, holding over 44% of the global IT Service Management (ITSM) market share (Gartner, 2024), and serves as the reference platform for this thesis. Its CMDB arranges CIs in a class hierarchy rooted at the `cmdb_ci` base table. Each subclass inherits base attributes (name, `sys_id`, operational status, environment) and adds domain-specific fields. For application inventory purposes, the most relevant class is `cmdb_ci_appl` (Application), which extends the base CI with attributes such as version, business unit, used-for classification, and references to the owning support group (ServiceNow, 2024a).

Beyond the application record itself, the CMDB captures relationships between CIs through a dedicated relationship table (`cmdb_rel_ci`). These relationships model dependencies such as “runs on” (application to server), “depends on” (application to database), and “hosted on” (application to cloud service). For the framework, the most valuable relationships are those linking business applications to technical assets that correspond to entities in the security data: repository names, deployment targets, and infrastructure components. ServiceNow also allows organizations to define custom attributes and relationship types, so the specific fields available for ingestion may vary between deployments.

1.1.3 Integration Options

Integrating CMDB data into an external data platform requires choosing between two strategies: pulling data from ServiceNow via its APIs, or pushing data from a custom ServiceNow application to an external target.

ServiceNow APIs

ServiceNow exposes two complementary Representational State Transfer (REST) APIs for CMDB data extraction. The Table API provides generic CRUD operations against any ServiceNow table through the endpoint `/api/now/table/{tableName}` (ServiceNow, 2024c). Responses are JavaScript Object Notation (JSON)-formatted, with support for server-side filtering (`sysparm_query`), field selection (`sysparm_fields`), and pagination (`sysparm_limit` combined with `sysparm_offset`, with a default maximum of 10 000 records per request). The `sysparm_display_value` parameter resolves reference fields to human-readable names instead of internal `sys_id` values.

The CMDB Instance API (`/now/cmdb/instance/{className}`) provides a CMDB-aware alternative that understands the CI class hierarchy (ServiceNow, 2024a). It can retrieve a CI with its relationships in a single call, reducing the number of API calls needed to reconstruct the application-to-asset mapping. However, it is more constrained in its query capabilities than the Table API.

Both APIs require authentication, typically through Basic Authentication with a service account granted the `rest_service` or `cmdb_read` role, or through OAuth 2.0 for more security-conscious deployments. Rate limits vary by instance configuration and are governed by platform transaction quotas rather than per-endpoint limits. For incremental data extraction, the `sys_updated_on` timestamp field present on all records serves as a reliable high-water mark, allowing consumers to query only records modified since the last extraction.

Any pull-based integration must handle authentication, pagination, rate limiting, and incremental state management. Several integration libraries and platform-native connectors abstract these concerns; the specific tooling choice depends on the target data platform and is discussed in subsection 3.2.1.

ServiceNow Application

An alternative to pulling data is to push it from the source. ServiceNow supports building scoped applications that run on the platform itself. A custom application can use Outbound REST Messages to send CMDB data to an external target (e.g., a cloud storage bucket or a REST endpoint) on a schedule defined through Scheduled Jobs or Flow Designer. The application can also expose a Scripted REST API (ServiceNow, 2024b), a custom endpoint that shapes the data exactly as the consumer needs it, pre-joining application records with their relationships and custom attributes in a single response.

The advantage is full access to ServiceNow's internal scripting capabilities, relationship traversal, and business logic without external API call overhead. The disadvantage is that development and maintenance happen inside the ServiceNow platform, requiring Servi-

ceNow development expertise and a separate deployment lifecycle. Changes to the CMDB schema or extraction logic must be coordinated across two platforms. This approach is best suited for organizations with mature ServiceNow development teams that prefer to own the data export logic on the source side.

Integration Challenges

The application inventory presents the most significant integration challenge among all data sources because it is the least standardized. Security testing tools converge around common API patterns and output formats, but each organization structures its application inventory differently: the hierarchy of applications, the granularity of asset mapping, and the attributes captured vary substantially. This makes the application inventory connector the hardest to generalize and the one most likely to require organization-specific customization.

Data quality in CMDBs is a persistent concern. Application records may be incomplete (missing ownership or criticality assignments), outdated (reflecting decommissioned applications still listed as active), or inconsistently maintained across business units. Any integration must account for these quality issues through validation rules and default values that surface problems without blocking the data pipeline.

Despite these difficulties, the application inventory is indispensable. Without the business context it provides, security findings lack the organizational meaning needed for effective prioritization and governance.

1.2 Software Development

1.2.1 Source Code Management

1.2.2 Continuous Integration and Delivery

1.2.3 Issue Tracking

1.3 Application Security

1.3.1 Static Application Security Testing

1.3.2 Software Composition Analysis

1.3.3 Secret Detection

1.3.4 Dynamic Testing and Penetration Testing

1.3.5 Container Image Scanning

1.3.6 Infrastructure as Code Security

1.3.7 Runtime Security Monitoring

1.4 Data Engineering

1.4.1 Data Platform Architecture

1.4.2 Data Integration Patterns

1.4.3 Domain Data Model

1.5 Related Work and Gap Analysis

1.5.1 Existing Approaches

1.5.2 Databricks Lakewatch

1.5.3 Comparative Analysis and Research Gap

2. Framework

2.1 Solution Architecture

2.1.1 Component Design

2.1.2 Medallion Architecture

2.1.3 Technology Stack

2.2 Data Model

2.2.1 Schema Patterns

2.2.2 Entity Model

2.2.3 Aggregation Model

2.3 Connector Framework

2.3.1 Connector Abstraction

2.3.2 Ingestion Patterns

2.3.3 Transformation Patterns

2.4 Analytics and Serving Layer

2.4.1 Aggregation Patterns

2.4.2 ML Workflows

2.4.3 Serving Patterns

2.5 Deployment and Engineering Practices

2.5.1 Deployment Strategy

2.5.2 Project Structure

3. Reference Implementation

3.1 Environment and Deployment

3.1.1 Workspace Setup

3.1.2 Silver Schema

3.1.3 Project Structure and CI/CD

3.2 Connectors

3.2.1 ServiceNow

3.2.2 GitHub

3.2.3 GitLab

3.2.4 SonarQube

3.2.5 Semgrep

3.2.6 Dependency-Track

3.2.7 TruffleHog

3.2.8 Vulnerability Enrichment

3.3 Analytics and Serving

3.3.1 Application-Repository Mapping

3.3.2 Application Risk Scoring

3.3.3 Remediation and Compliance Metrics

3.3.4 Vulnerability Trends

3.3.5 Risk Prediction Model

Conclusion

Thesis Outcomes and Contributions

Limitations

Future Work

References

AXELOS. (2019). *ITIL foundation: ITIL 4 edition*. TSO (The Stationery Office). (Cit. on pp. 12, 13).

Gartner. (2024). *Market share: IT service management, worldwide, 2024*. Retrieved April 11, 2026, from https://www.servicenow.com/blogs/2025/no-1-6-tech-workflow-segments (cit. on p. 13).

ServiceNow. (2024a). *CMDB instance API reference*. Retrieved March 7, 2026, from https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/cmdb-instance-api.html (cit. on pp. 13, 14).

ServiceNow. (2024b). *Scripted REST APIs*. Retrieved April 11, 2026, from https://www.servicenow.com/docs/bundle/yokohama-api-reference/page/integrate/custom-web-services/concept/c_CustomWebServices.html (cit. on p. 14).

ServiceNow. (2024c). *Table API reference*. Retrieved March 7, 2026, from https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html (cit. on p. 14).

Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on p. 12).

Appendices

A. Generative AI Use Disclosure

A.1 Text Content

A.2 Diagrams and Figures

A.3 Source Code

A.4 Requirements Specification